

```
.....Returning memoized result: 1
--snip--
Calling fibonacci(8) (nthNumber - 2).
.fibonacci(8) called.
.Returning memoized result: 21
Call to fibonacci(10) returning 55.
55
fibonacci(10) called.
Returning memoized result: 55
55
```

To memoize this function, we'll create a dictionary (in Python) or object (in JavaScript) in a global variable named `fibonacciCache` ❶. Its keys are the arguments passed for the `nthNumber` parameter, and its values are the integers returned by the `fibonacci()` function given that argument. Every function call first checks if its `nthNumber` argument is already in the cache. If so, the cached return value is returned ❷. Otherwise, the function runs as normal (though it also adds the result to the cache just before the function returns ❸ ❹).

The memoized function is effectively expanding the number of base cases in the Fibonacci algorithm. The original base cases are only for the first and second Fibonacci numbers: they immediately return 1. But every time a recursive case returns an integer, it becomes a base case for all future `fibonacci()` calls with that argument. The result is already in `fibonacciCache` and can be immediately returned. If you've already called `fibonacci(99)` once before, it becomes a base case just like `fibonacci(1)` and `fibonacci(2)`. In other words, memoization improves the performance of recursive functions with overlapping subproblems by increasing the number of base cases. Notice that the second time our program tries to find the 10th Fibonacci number ❺, it immediately returns the memoized result: 55.

Keep in mind that while memoization can reduce the number of redundant function calls a recursive algorithm makes, it doesn't necessarily reduce the growth of frame objects on the call stack. Memoization won't prevent stack overflow errors. Once again, you may be better off forgoing a recursive algorithm for a more straightforward iterative one.